CAD for VLSI

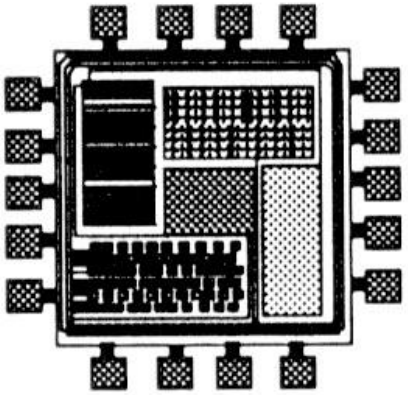
Floorplanning 1

Hierarchical Design

- Several blocks after partitioning:

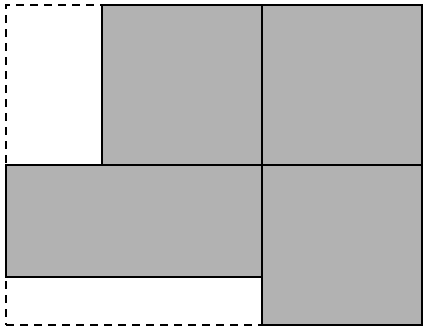
- Need to:
  - Put the blocks together
  - Design each block

  Which step to go first?



Hierarchical Design

- How to put the blocks together without knowing their shapes and the positions of the I/O pins?
- If we design the blocks first, those blocks may not be able to form a tight packing



Floorplanning

- The floorplanning problem is to plan the positions and shapes of the modules at the beginning of the design cycle to optimize the circuit performance…
  - chip area
  - total wirelength
  - delay of critical path
  - routability
  - others, ex: noise, heat dissipation, …

Floorplanning vs. Placement

- Both determines block positions to optimize the circuit performance.
- Floorplanning:
  - Details like shapes of blocks, I/O pin positions, etc. are not yet fixed (blocks with flexible shape are called soft blocks)
- Placement:
  - Details like module shapes and I/O pin positions are fixed (blocks with no flexibility in shape are called hard blocks)

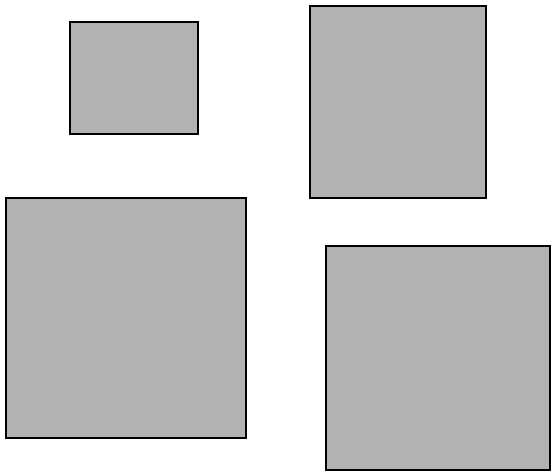
Floorplanning Problem

- Input:
  - n Blocks with areas $A_1, \ldots, A_n$
  - Bounds r_i and s_i on the aspect ratio of block B_i
- Output:
  - Coordinates (x_i, y_i), width w_i and height h_i for each block such that $h_i w_i = A_i$ and $r_i \leq h_i/w_i \leq s_i$
- Objective:
  - To optimize the circuit performance

Bounds on Aspect Ratios

- If there is no bound on the aspect ratios, can we pack everything tightly?

Sure!

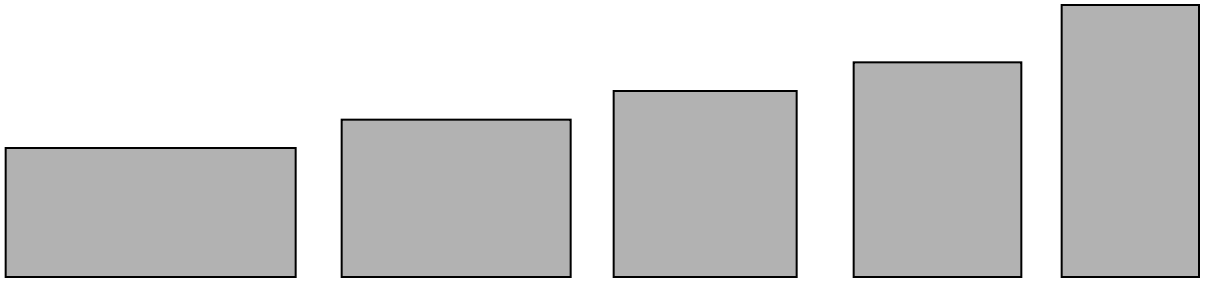




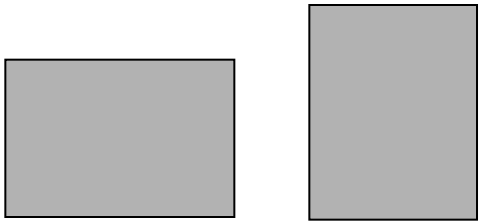
- But we don't want to layout blocks as long strips, so we require $r_i \leq h_i/w_i \leq s_i$ for each i

Bounds on Aspect Ratios

- We can also allow several shapes for soft blocks:



- For hard blocks, the orientations can be changed:



Objective Function

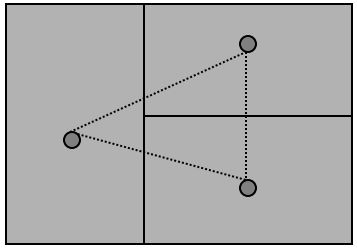
- A commonly used objective function is a weighted sum of area and wirelength

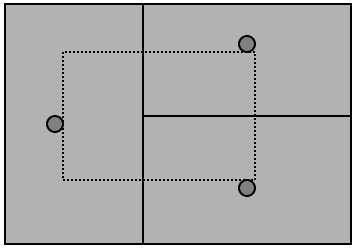
$$\text{cost} = \alpha A + \beta L$$

  - A is the total area of the packing
  - L is the total wirelength
  - α and β are constants

Wirelength Estimation

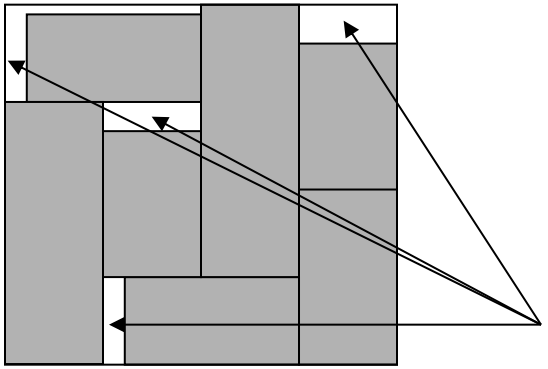
- Exact wirelength of each net is not known until routing is done
- In floorplanning, even pin positions are not known yet.
- Some possible wirelength estimations:
  - Center-to-center estimation
  - Half-perimeter estimation





Dead space

- Dead space is the space that is wasted:

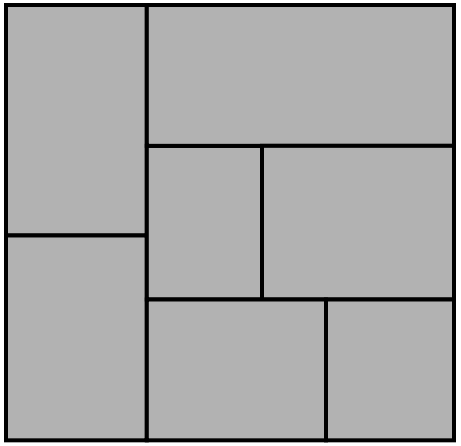


- Minimizing area is the same as minimizing dead space
- Dead space percentage is computed as

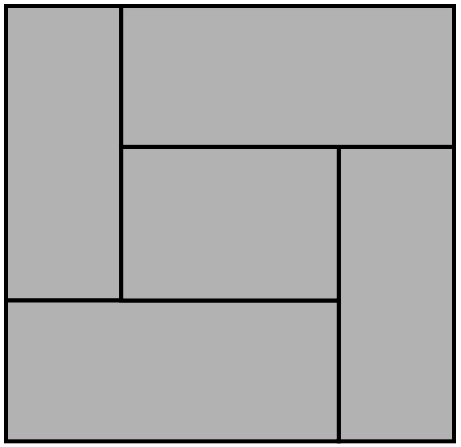
$$(A - \Sigma_i A_i) / A \times 100\%$$

Slicing and Non-Slicing Floorplan

- Slicing Floorplan:
  - One that can be obtained by repetitively subdividing (slicing) rectangles horizontally or vertically

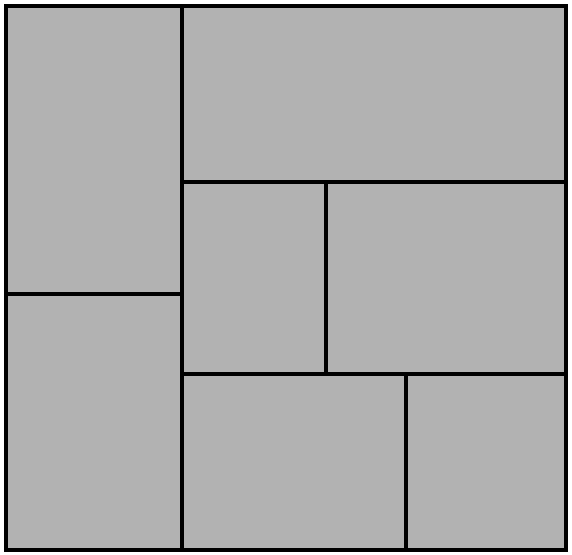


- Non-Slicing Floorplan:
  - One that may not be obtained by repetitively subdividing alone

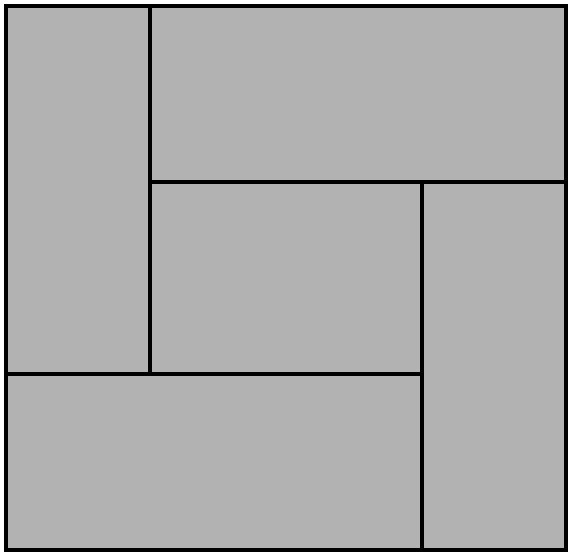


- Otten (LSSS-82) pointed out that slicing floorplans are much easier to handle

NTUST Representation?



Slicing Floorplan



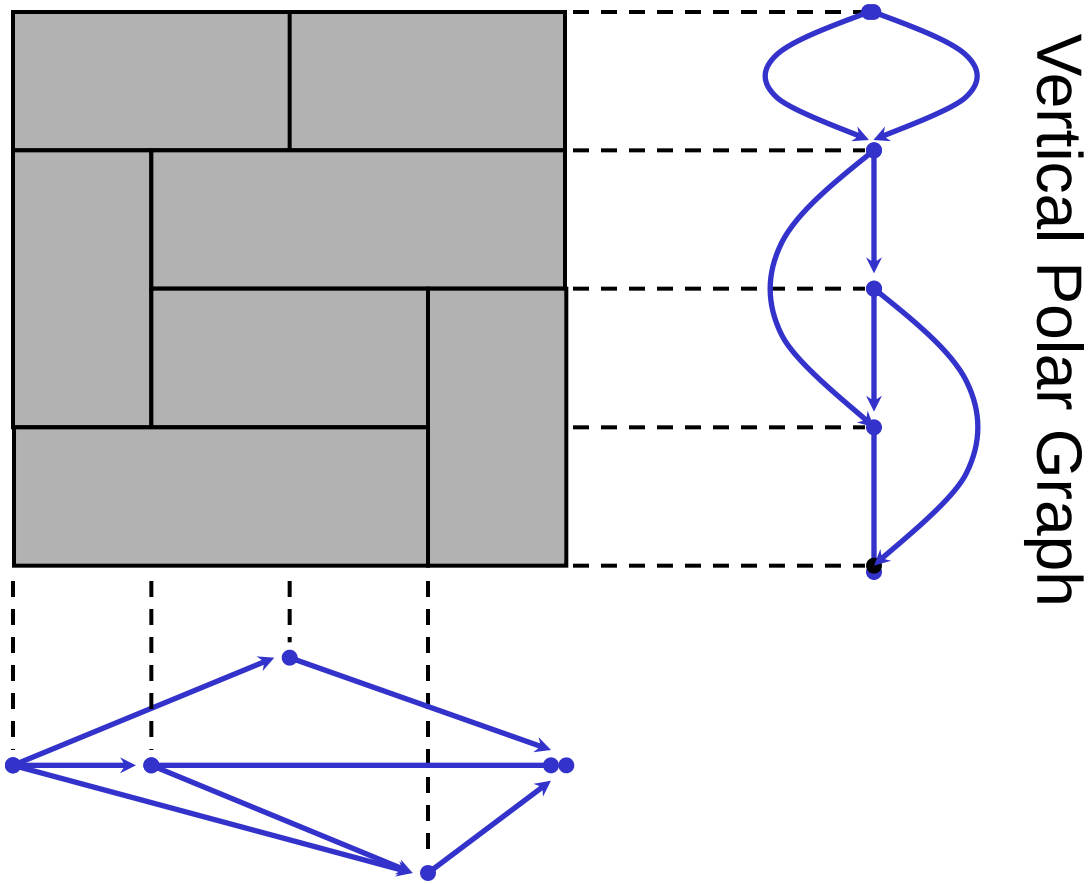
Non-slicing Floorplan

Polar Graph Representation

- A graph representation of floorplan
- Each floorplan is modeled by a pair of directed acyclic graphs:
  - Horizontal polar graph
  - Vertical polar graph
- For horizontal (vertical) polar graph,
  - Vertex: Vertical (horizontal) channel
  - Edge: 2 channels are on 2 sides of a block
  - Edge weight: Width (height) of the block

- Note: There are many other graph representations

Polar Graph: Example

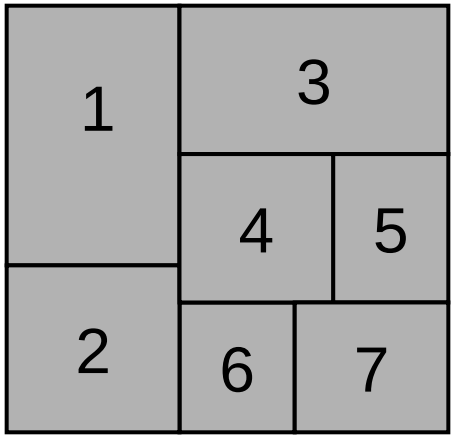


Simulated Annealing using Polish Expression Representation

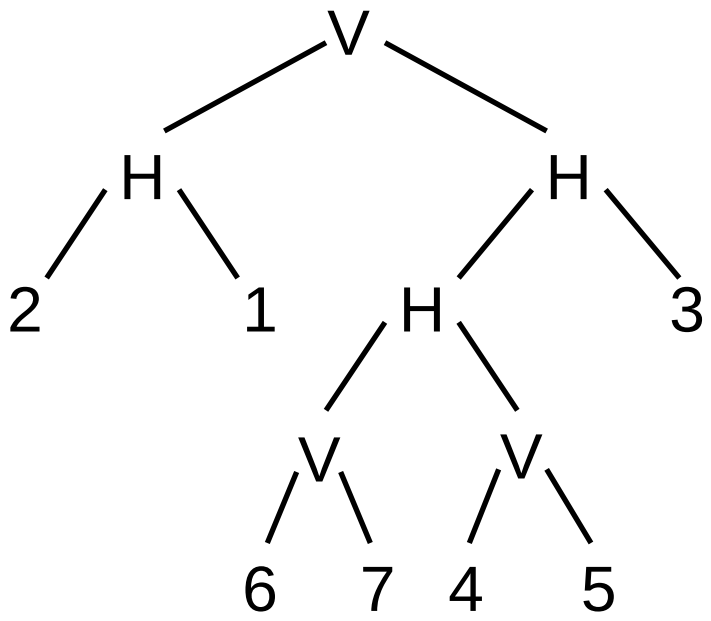
D.F. Wong and C.L. Liu,
"A New Algorithm for Floorplan Design"
DAC, 1986, pages 101-107.

Representation of Slicing Floorplan

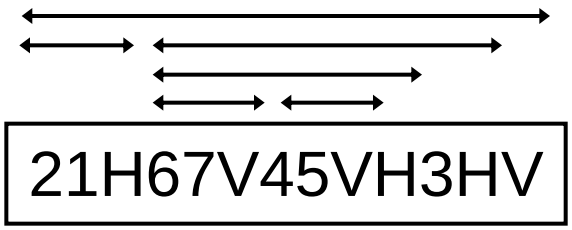
Slicing Floorplan



Slicing Tree



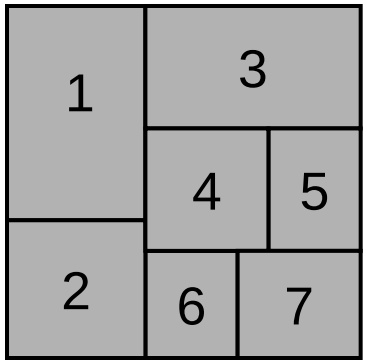
Polish Expression
(postorder traversal
of slicing tree)

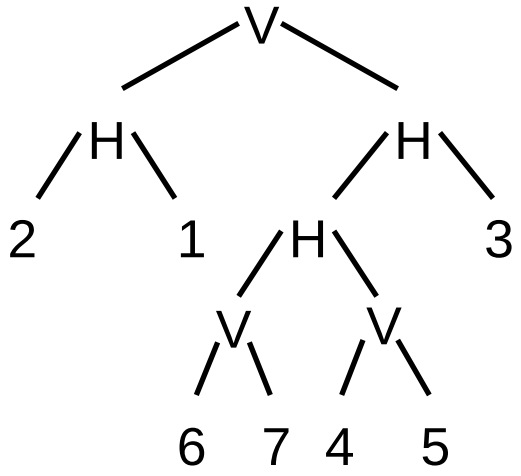


Polish Expression

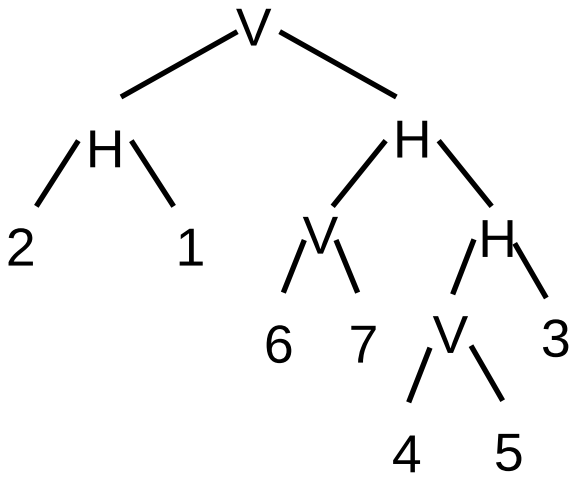
- Succinct representation of slicing floorplan
  - Roughly specifying relative positions of blocks
- Postorder traversal of slicing tree
  1. Postorder traversal of left sub-tree
  2. Postorder traversal of right sub-tree
  3. The label of the current root
- For *n* blocks, a Polish Expression contains *n* operands (blocks) and *n-1* operators (H, V)
- However, for a given slicing floorplan, the corresponding slicing tree (and hence polish expression) is not unique. Therefore, there is some redundancy in the representation

Redundancy of Polish Expression





21H67V45VH3HV

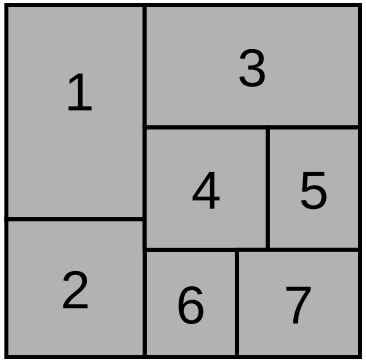


21H67V45V3HHV

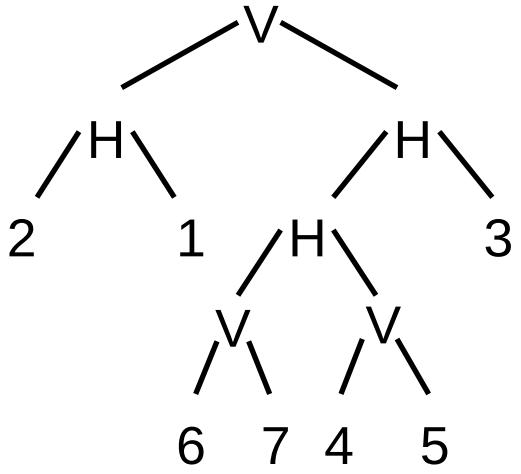
Skewed ST and Normalized PE

- Skewed Slicing Tree:
  - no node and its right son are the same.
- Normalized Polish Expression:
  - no consecutive H's or V's.

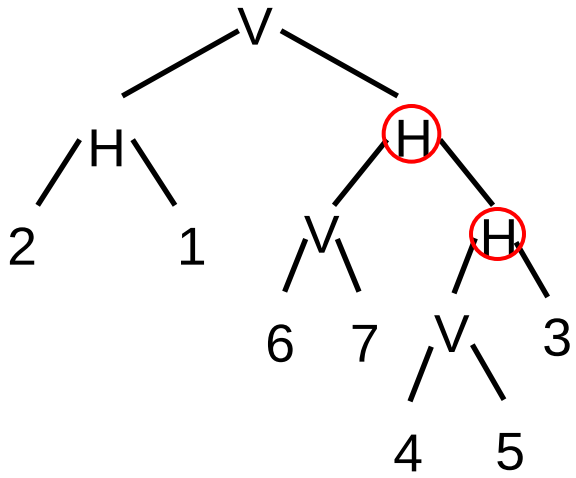
Slicing Floorplan



Slicing Tree (Skewed)



Slicing Tree



Polish Expression

21H67V45VH3HV

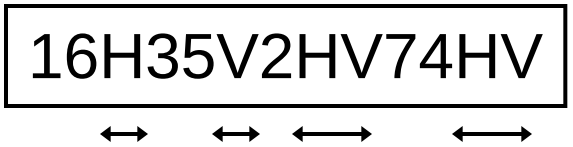
21H67V45V3HHV

Normalized Polish Expression

- There is a 1-1 correspondence between Slicing Floorplan, Skewed Slicing Tree, and Normalized Polish Expression
- Can we use Normalized Polish Expression to represent slicing floorplans?
  - What is a valid NPE?
- We formulate as a state space search problem

Neighborhood Structure

- Chain: HVHVH… or VHVHV…

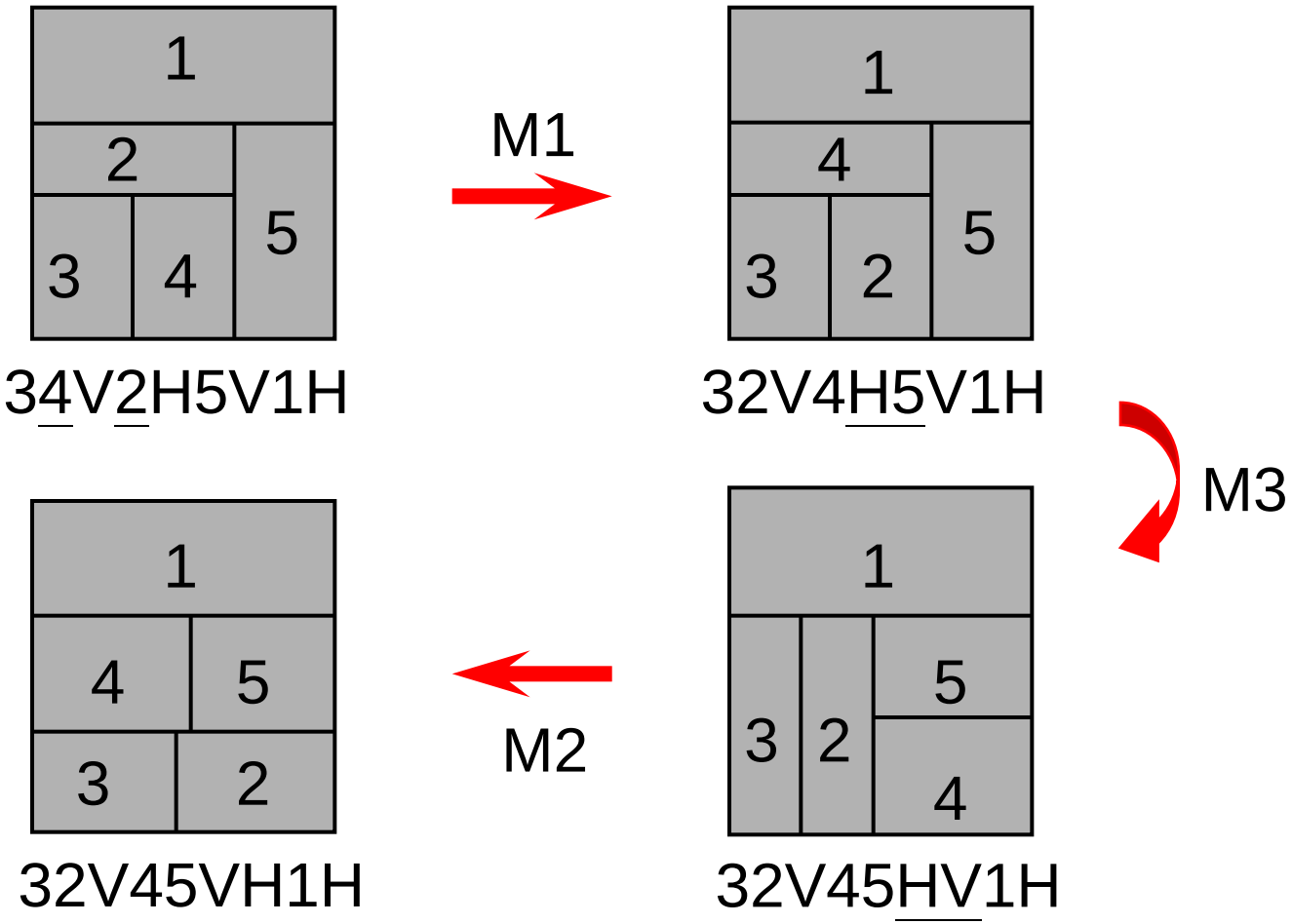


Chains

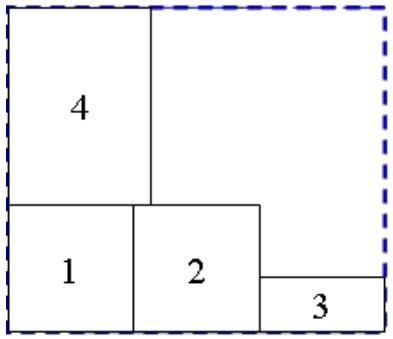
- The moves:
  - M1: Swap adjacent operands (ignoring chains)
  - M2: Complement some chain
  - M3: Swap 2 adjacent operand and operator
    - M3 can give you some invalid NPE. Checking for validity after M3 is needed

- It can be proved that every pair of valid NPE are connected

Example of Moves

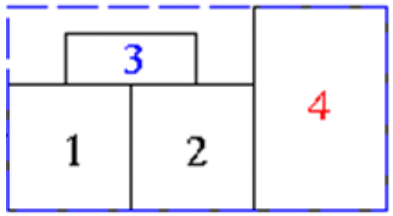


Example of Moves



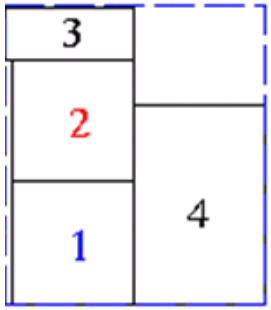
12V4H3V

M1



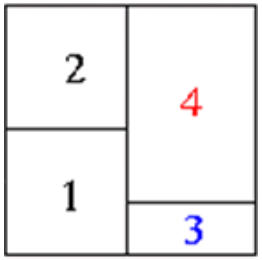
12V3H4V

M2



12H3H4V

M3

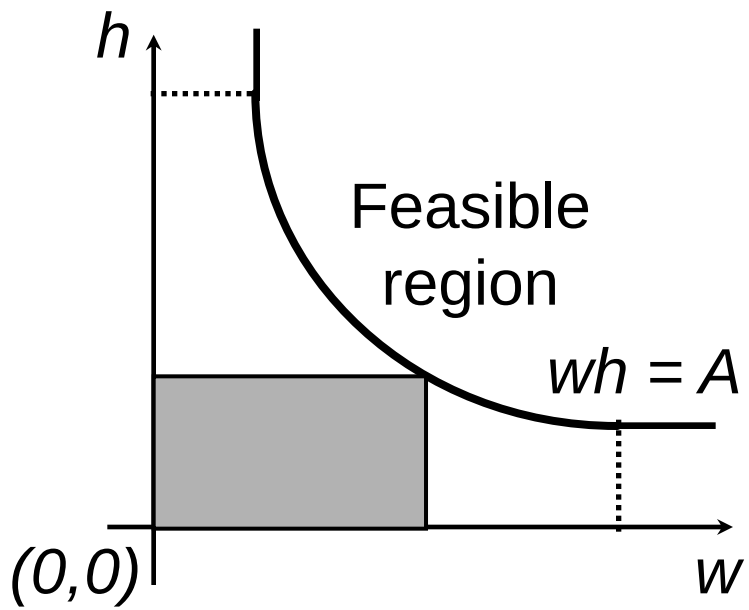


12H34HV

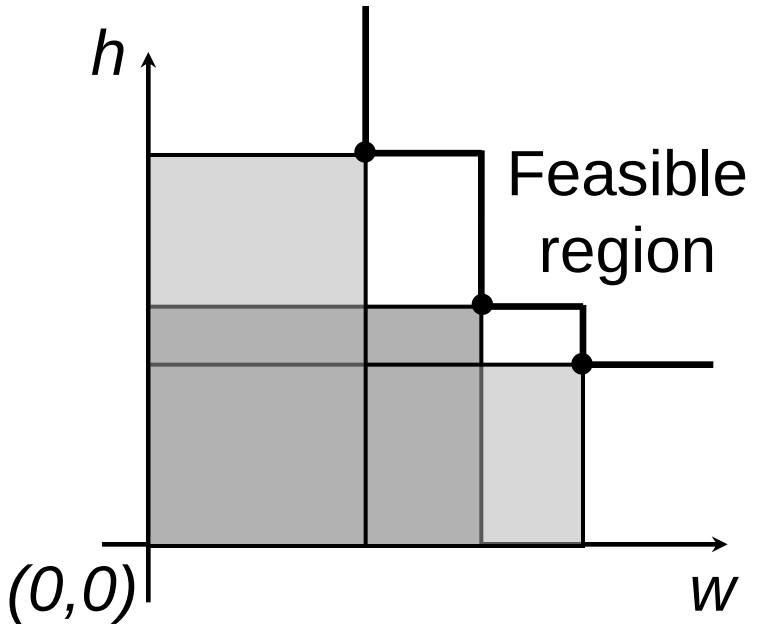
Shape Curve

- To represent the possible shapes of a block

Soft block

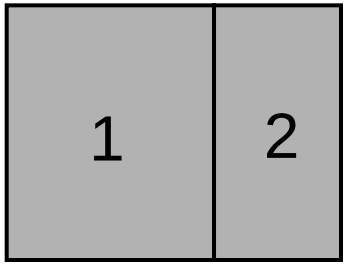


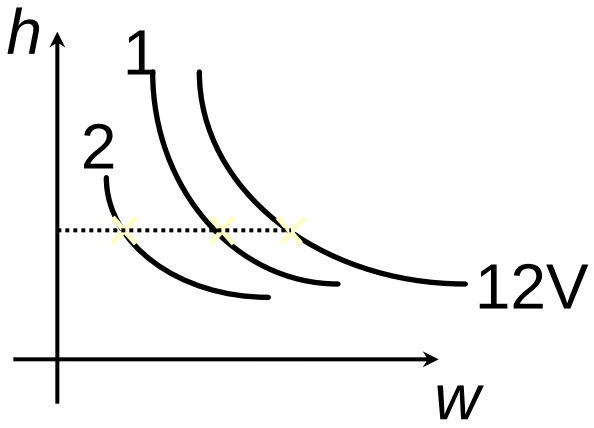
Block with several existing design



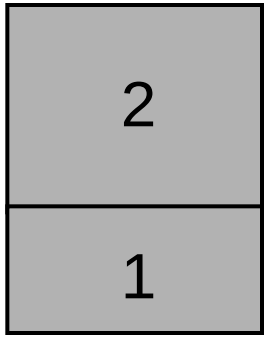
Combining Shape Curves

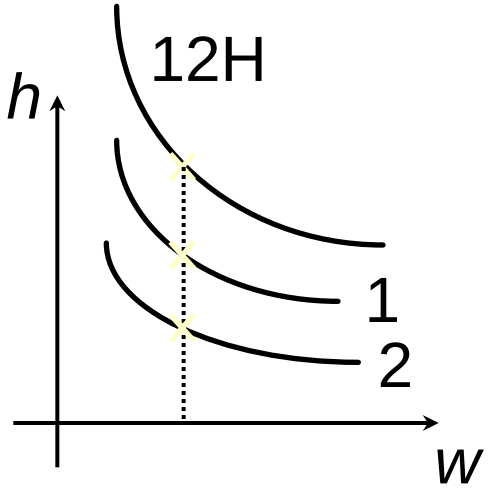
- 12V:



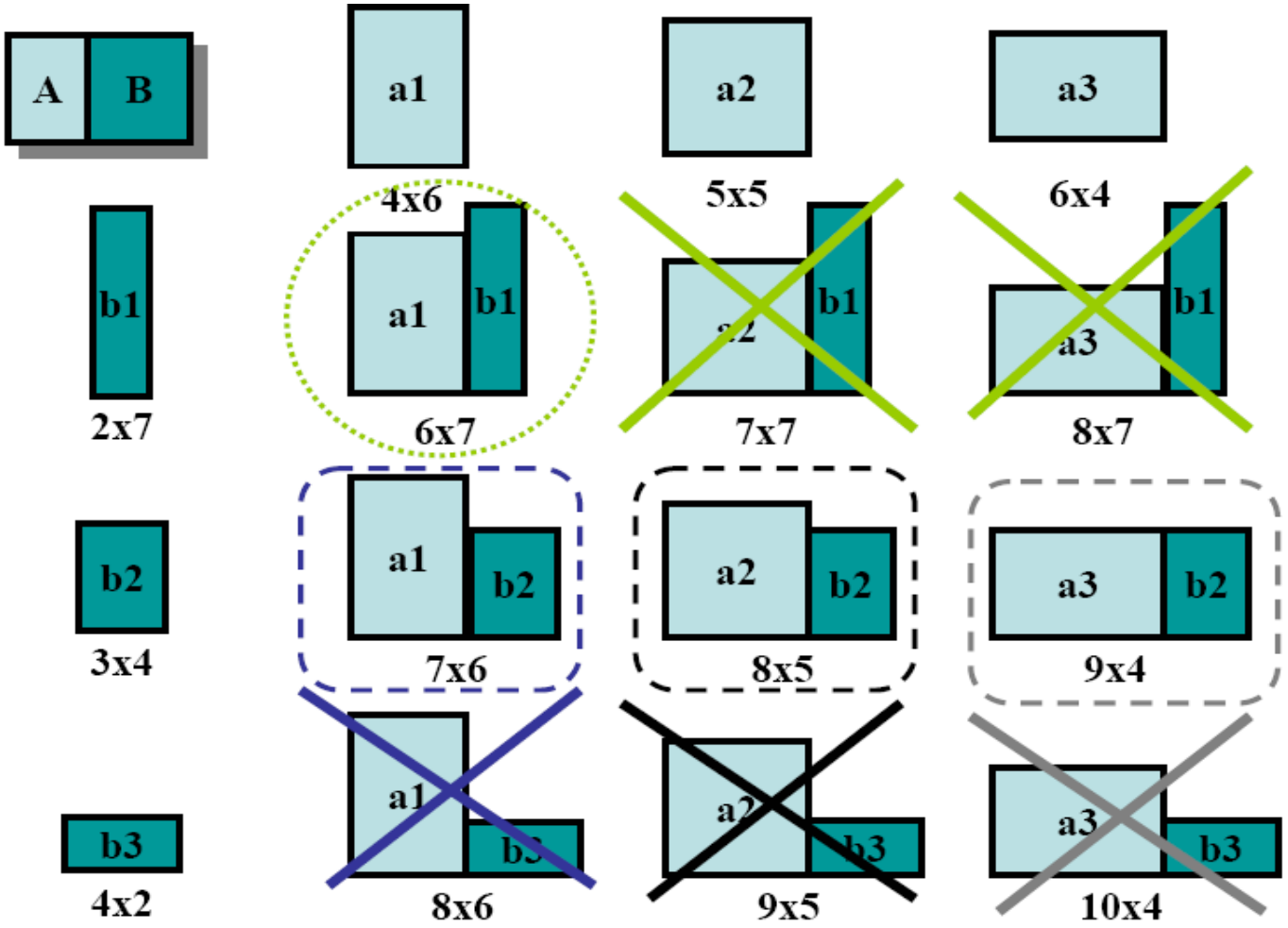


- 12H:



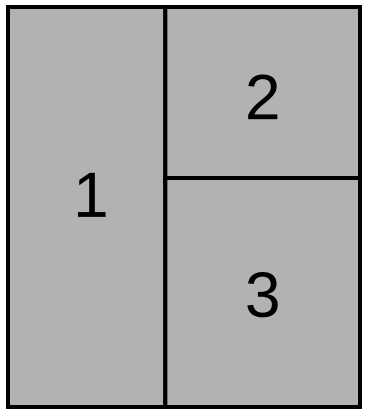


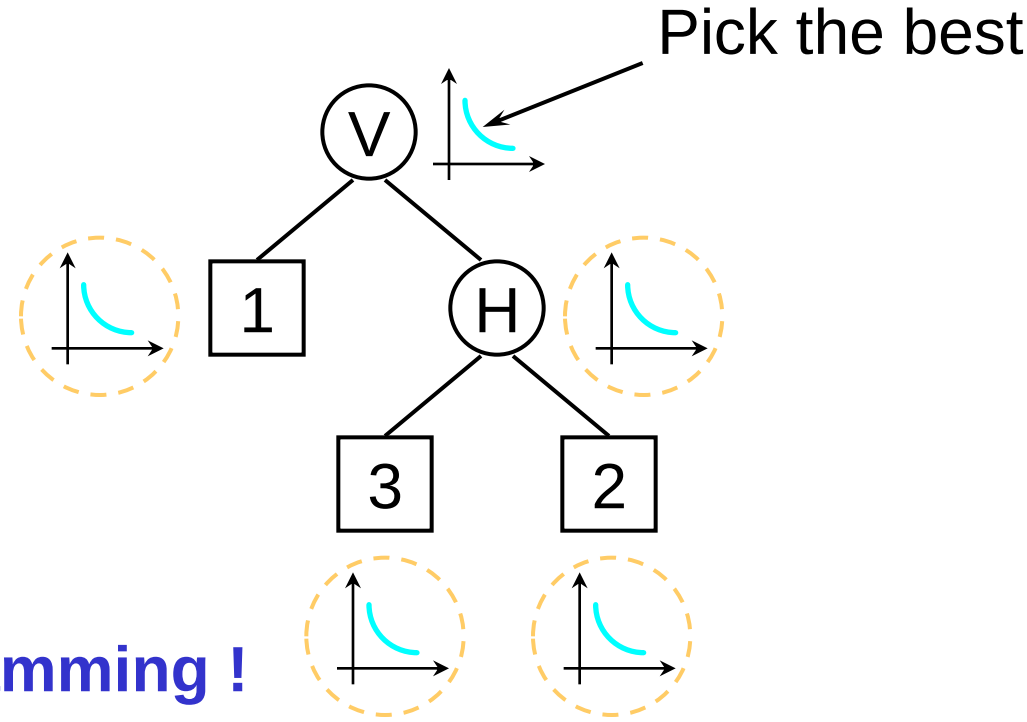
Example of Shape Curve Combining



Find the Best Area for a NPE

- Recursively combining shape curves



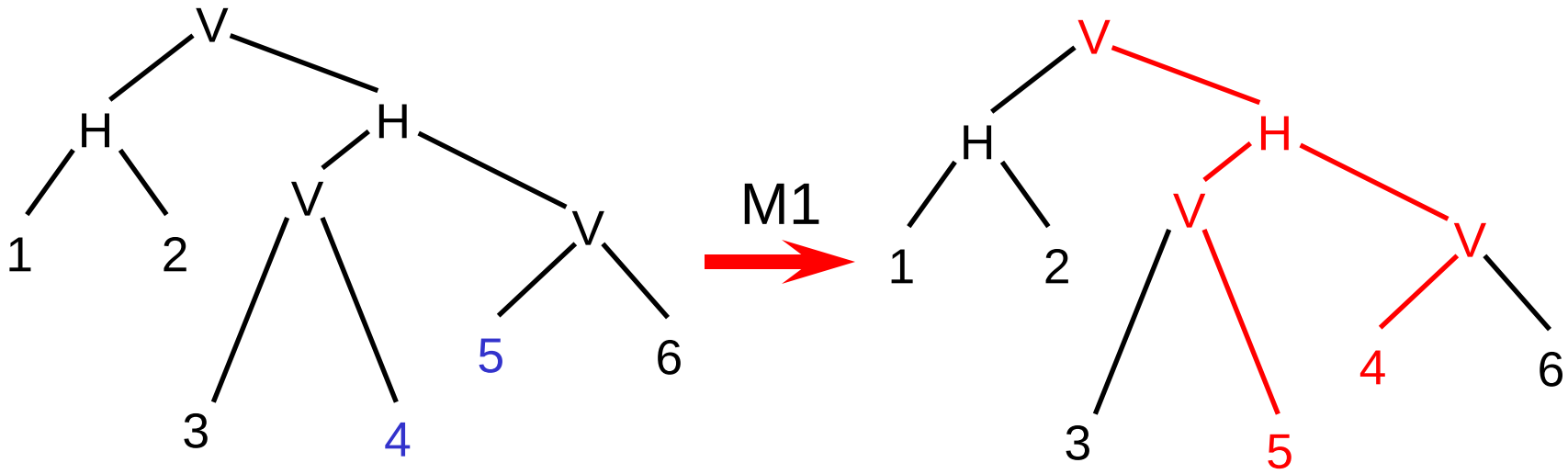


Dynamic Programming !

Updating Shape Curves after Moves

- If keeping k points for each shape curve, time for shape curve computation for each NPE is O(kn)
- After each move, there is only small change in the floorplan. So there is no need to start shape curve computation from scratch
- We can update shape curves incrementally after each move
- Run time is about O(k log n)

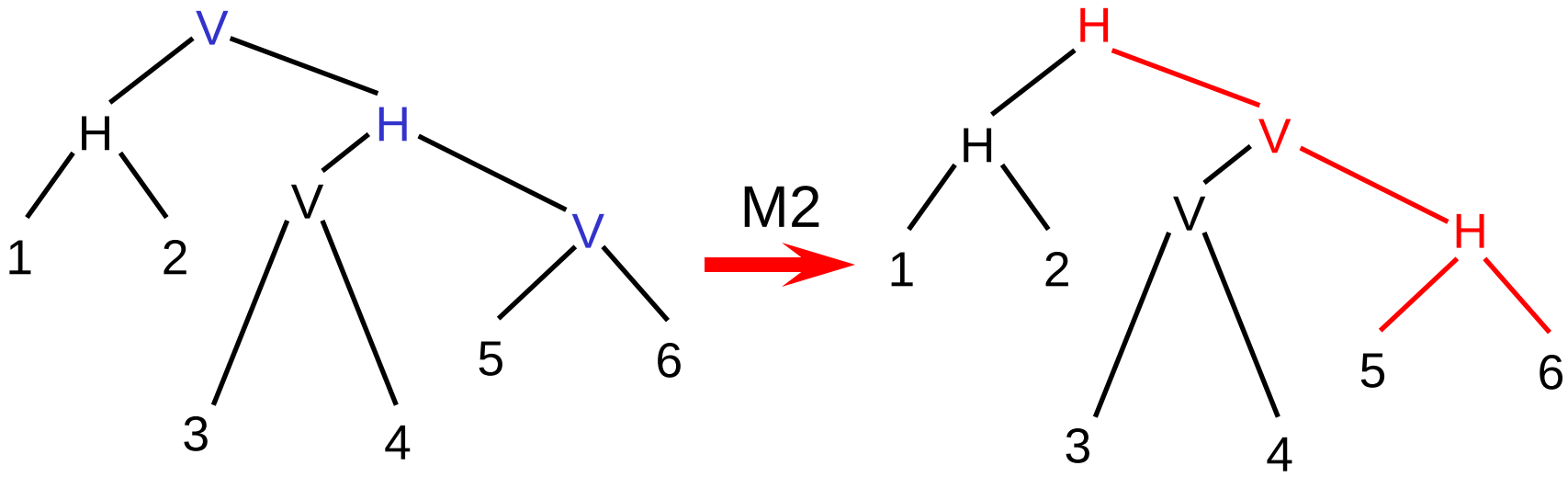
Example of Shape Curve Updating



12H34V56VHV

12H35V46VHV

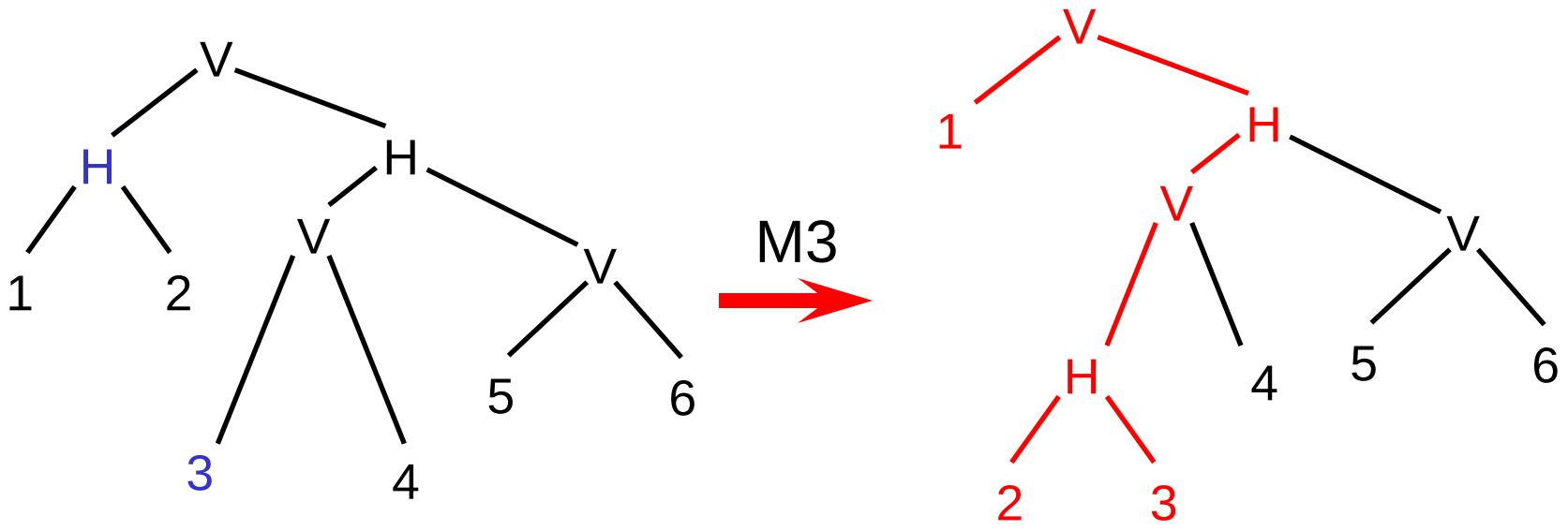
Example of Shape Curve Updating



12H34V56VHV

12H34V56HVH

Example of Shape Curve Updating

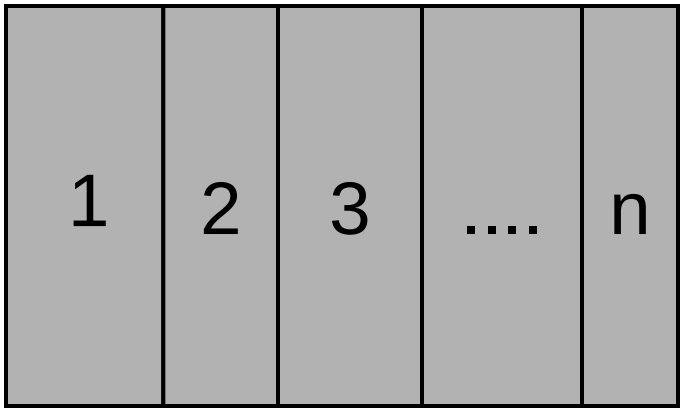


12H34V56VHV

123H4V56VHV

Initial State

- 12V3V4V...nV



Annealing Schedule

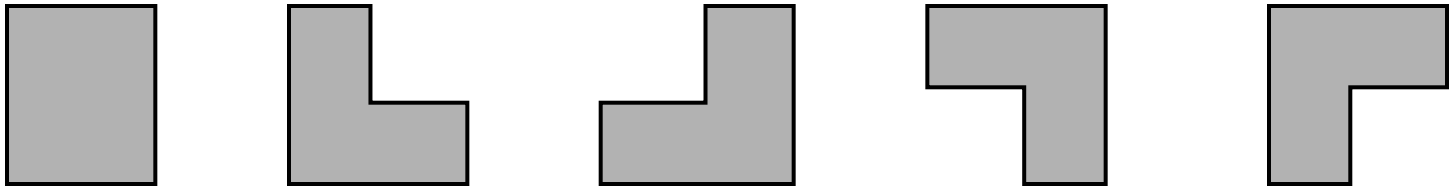
- $T_i = \alpha T_{i-1}$ where α=0.85
- At each temperature, try k $\times$ n moves
  - k is around 5 to 10
- Terminate the annealing process if
  - either # of accepted moves < 5%
  - or the temperature is low enough

Handling both Rectangular and L-Shaped Blocks

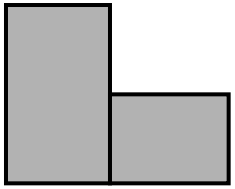
D.F. Wong and C.L. Liu,
"Floorplan Design for Rectangular and L-Shaped Modules"
ICCAD, 1987, pages 520-523.

Rectangular and L-Shaped Blocks

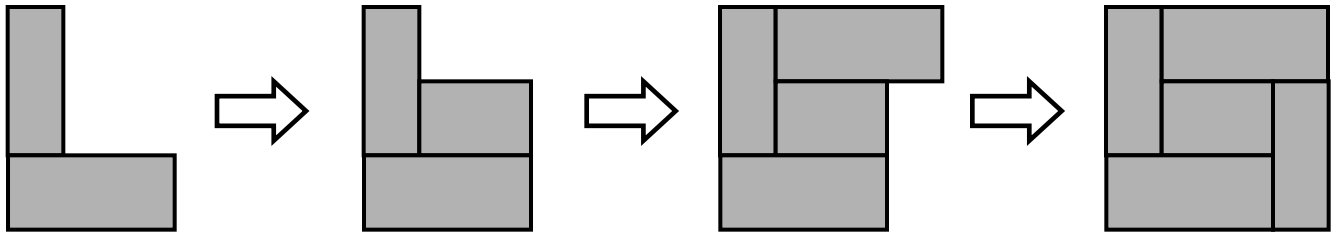
- Possible shapes:



- Note that L-shaped blocks can be produced even if we start with rectangular blocks only



- Can even generate non-slicing floorplans

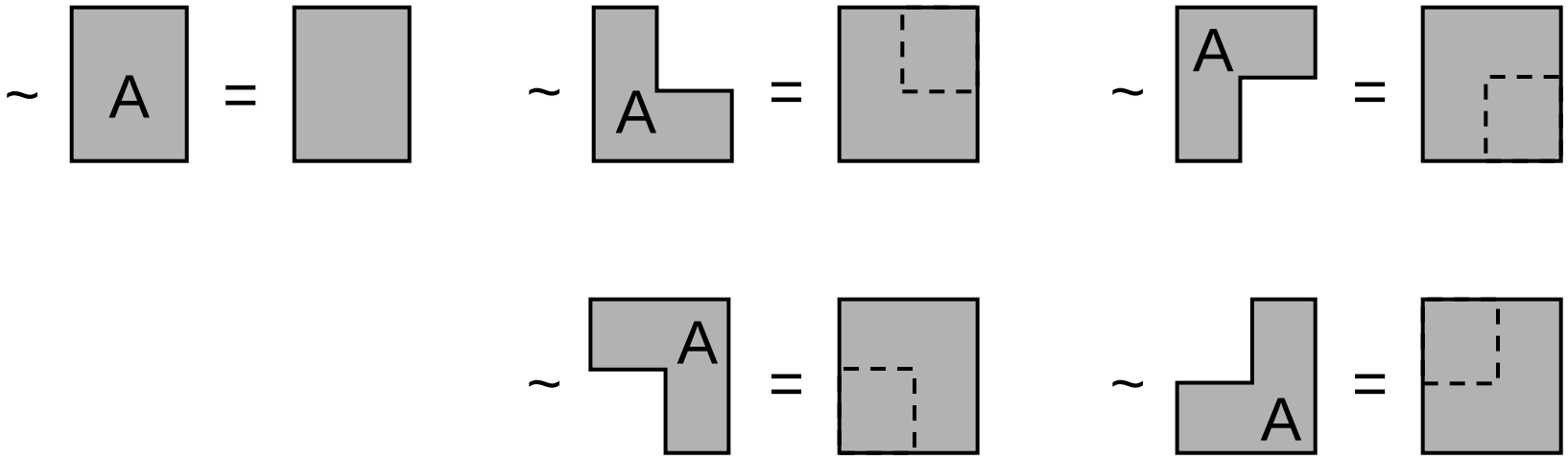


Basic Idea

- Similar to the DAC-86 paper by Wong & Liu:
  - Polish Expression representation
  - Simple moves to locally modify floorplan.
  - Simulated Annealing
- Differences from the DAC-86 paper:
  - 5 operators and 4 moves defined to handle the more complex shapes
  - Idea of shape curves no longer applicable.
  - Depend on Simulated Annealing to pick different shapes for blocks probabilistically

Operators

- 5 operators: ~, V1, V2, H1, H2
- Completion of A (~A):



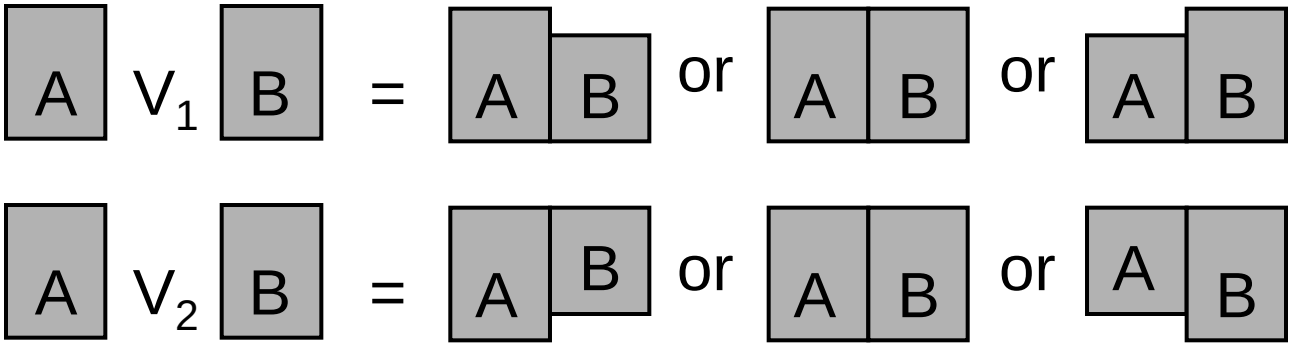
Binary Operators V_1, V_2, H_1 and H_2

- Need to define what "A op B" means,

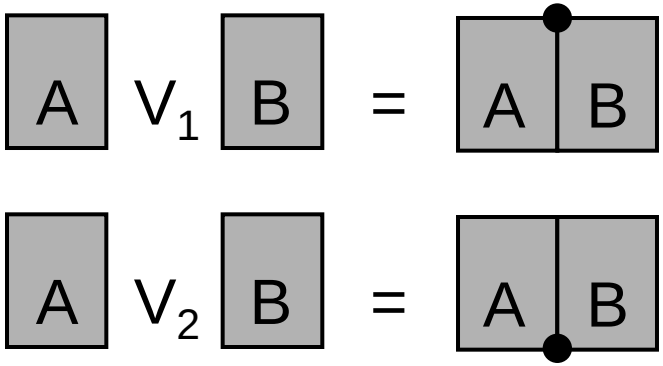
  where A and B are rectangular or L-shaped blocks, op is V_1, V_2, H_1 or H_2.
- Total # of ways to combine 2 blocks

  = 5 x 4 x 5 = 100

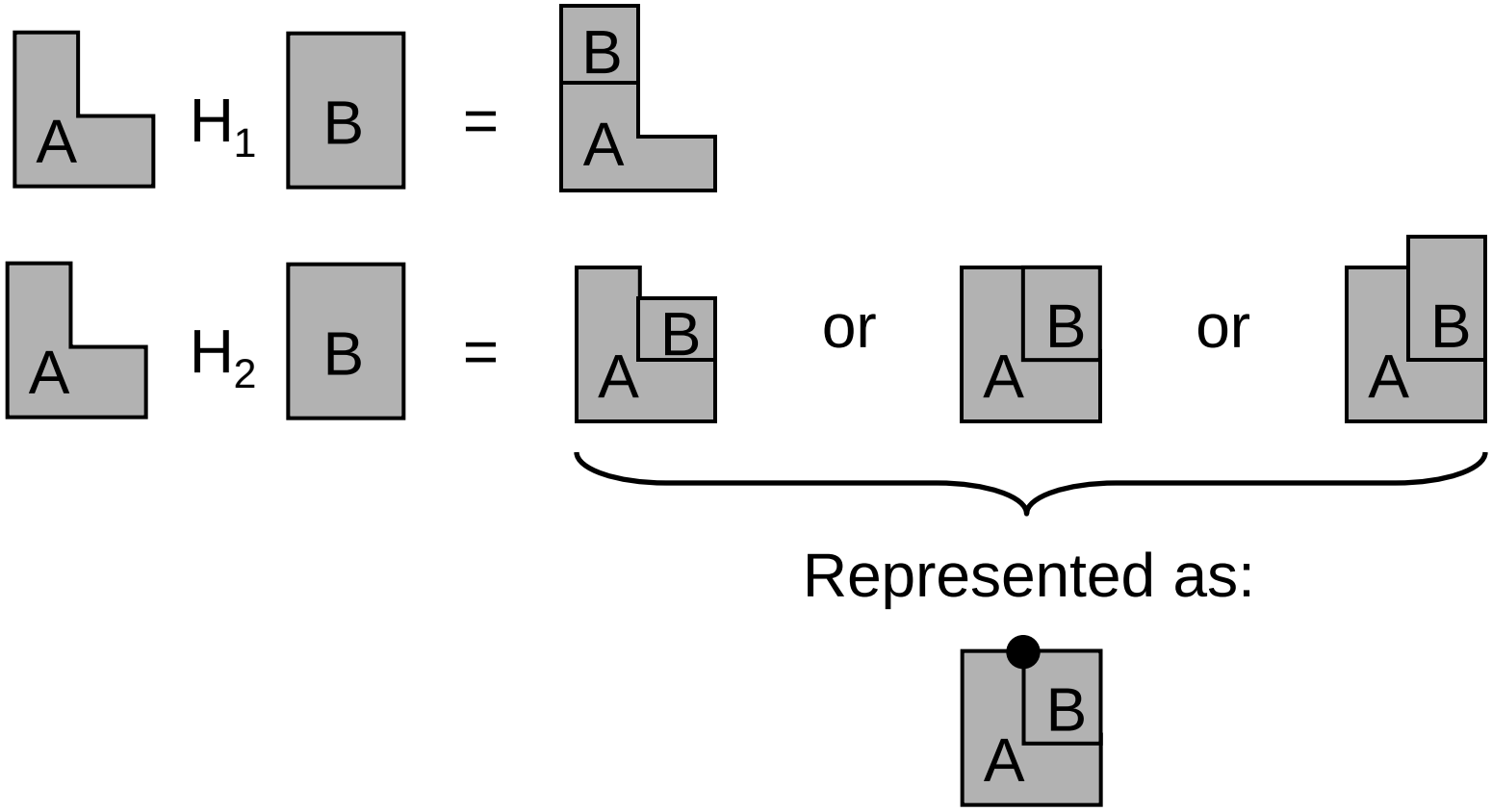
Example of Combining 2 Blocks



- Several possible outcomes. Represented as:



Another Example of Combining



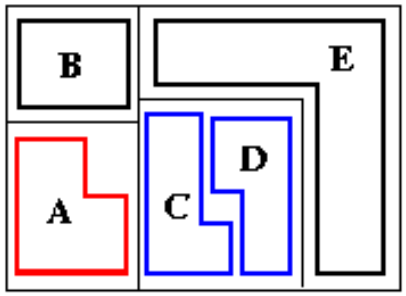
Moves

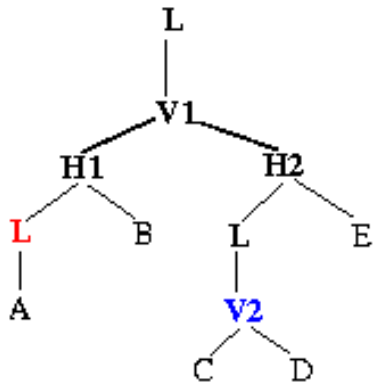
- Write the Polish Expression in the form:

  $b_1 u_1 b_2 u_2 \ldots b_{2n-1} u_{2n-1}$

  where b_i's are the n blocks, or the n-1 binary operators, and each u_i is either ~ or the empty string ε
- The moves:
  - M1: Modify for some I
    - change to a different shape or a different binary operator
  - M2: Change u_i to ~ or ε for some I
  - M3: Swap 2 blocks b_i and b_j
  - M4: Swap b_i and b_{i+1} for some i
    - M4 can obtain invalid PE (Checking needed)

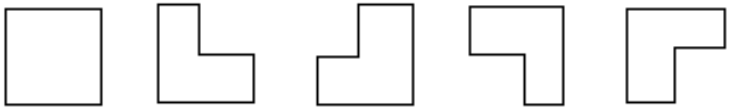
L-shaped Example



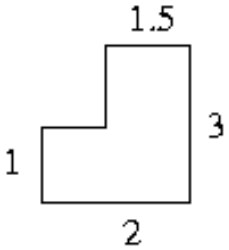


E = A L B H1 C D V2 L E H2 V1 L

Orientation:



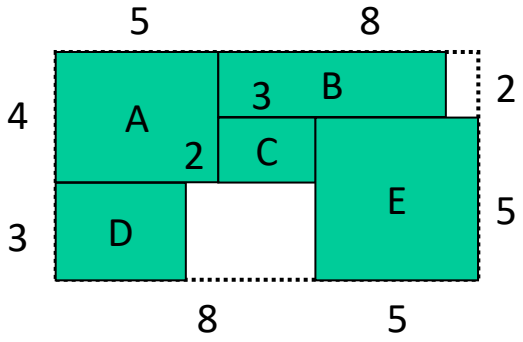
Representation:



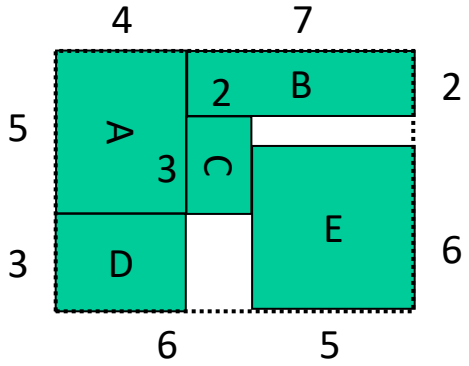
(2, 1.5, 3, 1)

Another Classic Work

- "Optimal Orientation of Cells in Slicing floorplan Designs"
  - L. Stockmeyer, Information and Control 57(1983), 91-101
  - This is an earlier paper than [Wong-Liu'86], dealing with simpler problem
  - Given slicing structure and a set of module shapes (or shape list)
  - How to orient these modules such that the total area is smallest?

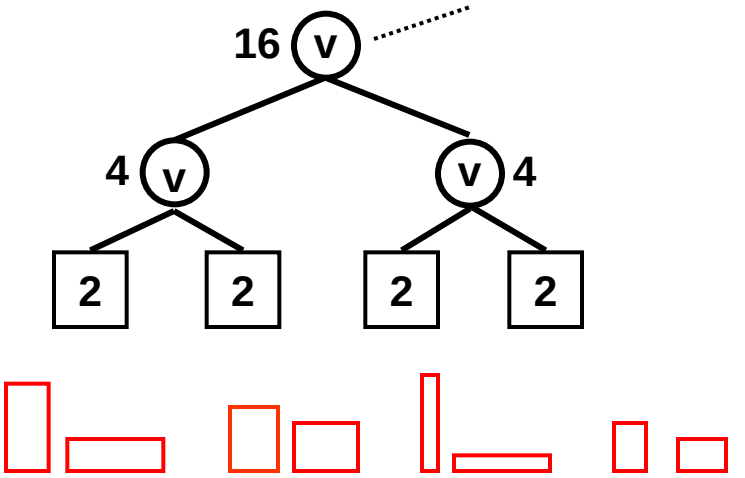


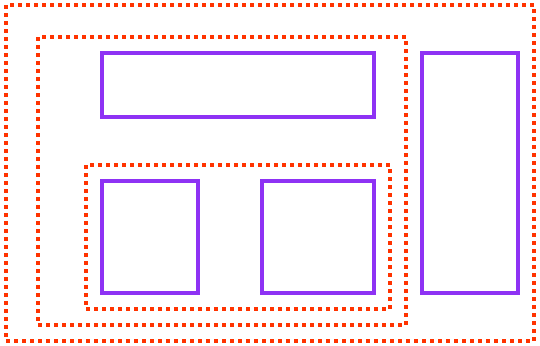
7x13

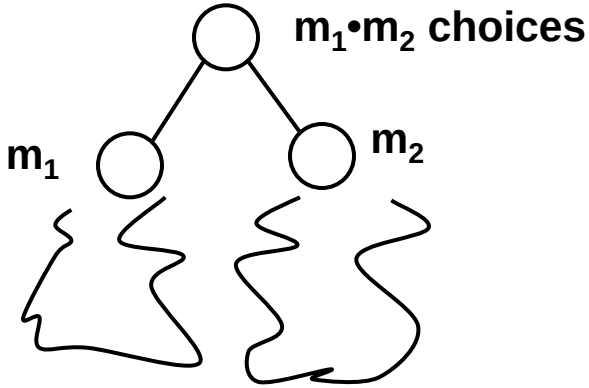


8x11

Difficulty



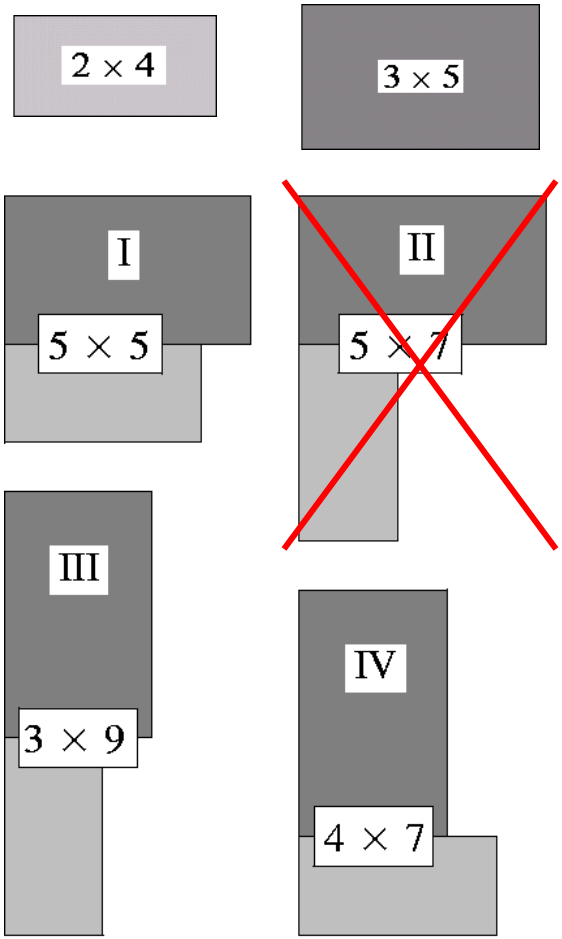


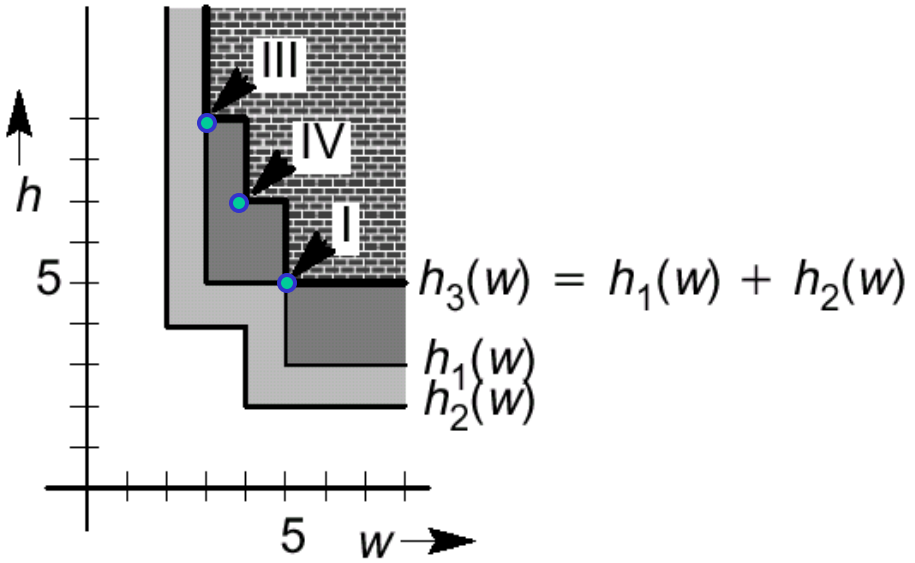


Key Idea

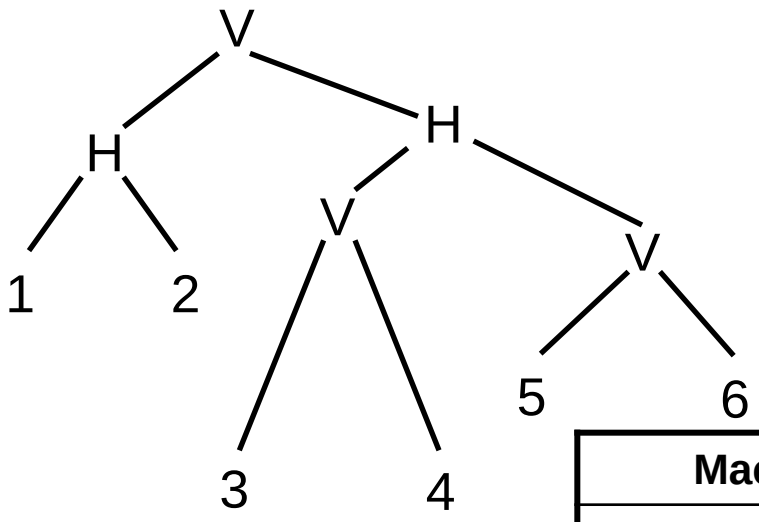
- Dynamic programming
  - Compute a set of irredundant solutions at each sub-tree rooted from the list of irredundant solutions for its two child subtrees
  - Pick the best solution from the list of irredundant solutions at the root

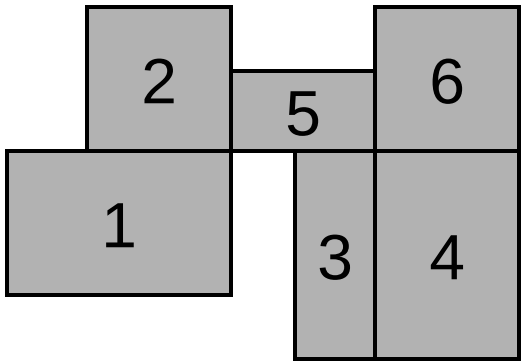
Example of Merging: Keep Irredundant Solutions Only





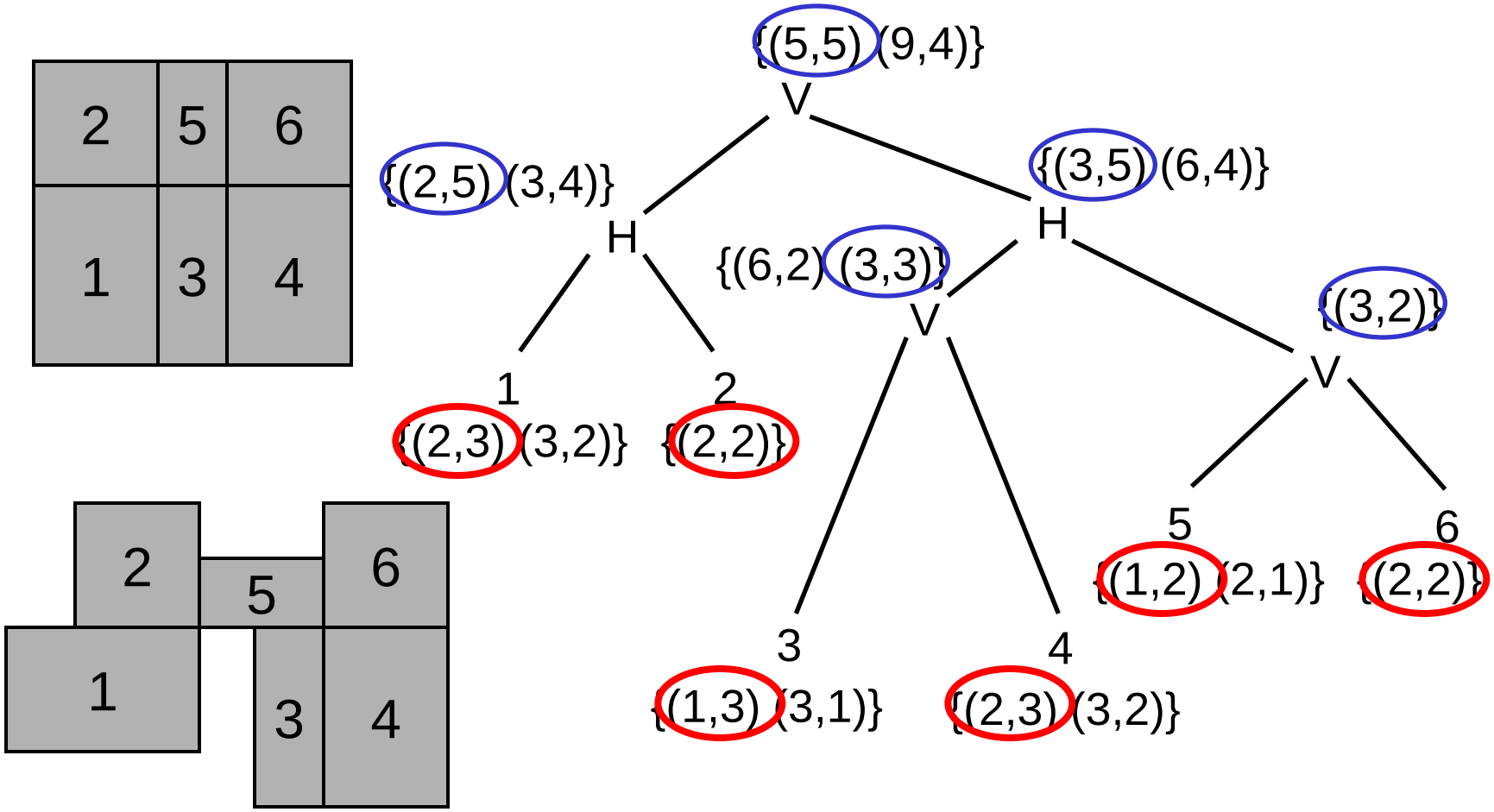
Example of Merging





Macro	Width	Height
1	3	2
2	2	2
3	1	3
4	2	3
5	2	1
6	2	2

Example of Merging



Stockmeyer Algorithm

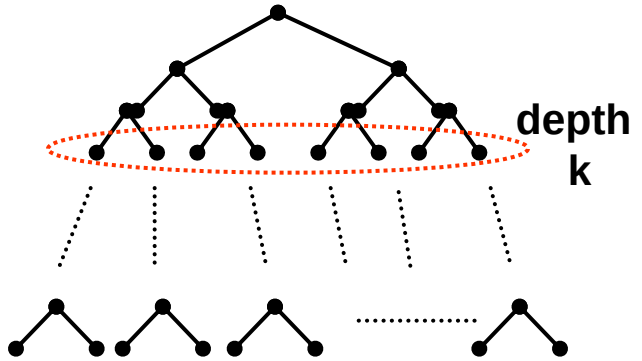
- **Phase 1: bottom-up**
  - Input: floorplan tree, modules shapes
  - Start with sorted shapes lists of modules
  - Perform Vertical_Node_Sizing & Horizontal_Node_Sizing
  - When get to the root node, we have a list of shapes. Select the one that is best in terms of area
- **Phase 2: top-down**
  - Traverse the floorplan tree and set module locations

Stockmeyer Algorithm

Procedure Vertical_Node_Sizing
Input: Sorted lists L = $\{(a_1, b_1), \ldots, (a_s, b_s)\}$, R = $\{(x_1, y_1), \ldots, (x_t, y_t)\}$,
where $a_i < a_j$, $b_i > b_j$, $x_i < x_j$, $y_i > y_j$ (for all $i < j$)
Output: A sorted list H = $\{(c_1, d_1), \ldots, (c_u, d_u)\}$,
where $u \leq s + t - 1$, $c_i < c_j$, $d_i > d_j$ (for all $i < j$)
Begin
H := Ø
i := 1, j := 1, k = 1
while ($i \leq s$) and ($j \leq t$) do
$(c_k, d_k) := (a_i + x_j, \max(b_i, y_j))$
H := H U $\{(c_k, d_k)\}$
k := k + 1
if $\max(b_i, y_j) = b_i$ then i := i + 1
if $\max(b_i, y_j) = y_j$ then j := j + 1

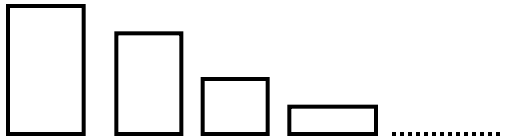
Complexity of the Algorithm

- *n*= # of leaves = 2 * # of modules
  - *d*=depth of the tree
  - Running time= O(*nd*)
  - Storage = O(*n*)



- at depth k,
  - sum of the lengths of the lists =O(*n*)
  - time to construct these lists =O(*n*)
  - configurations stored at this node can be release as soon as the node is processed

- Extension
  - Each module has *k* possible shapes
  - Running time and storage O(*nkd*)



Summary: What's BIG idea?

- Floorplan problem is definitely NP hard
- How to represent it compactly is a big deal
- Slicing is easier to deal with, so let's start with it
- Polish expression is very elegant and easy to make new moves
  - Need to be unique (NPE)
  - Bounding curve for area computation when merging two blocks, which can be computed incrementally
- For a given set of modules and the slicing tree, Stockmeyer's algorithm can give the optimal solution
  - But it's a very ideal situation that doesn't happen often ☹
  - Nice algorithm using dynamic programming

